Dados Estruturados, Ponteiros e Uso de Função com Ponteiro

Tipos estruturados

Em C, podemos definir um tipo de dado cujos campos são compostos por vários valores de tipos mais simples.

Para ilustrar, considere o desenvolvimento de programas que manipulam pontos no plano cartesiano. Cada ponto pode ser representado por suas coordenadas x e y, ambas representadas por valores reais.

Sem um mecanismo para agrupar essas duas informações, teríamos que representar cada ponto utilizando duas variáveis independentes.

```
float x;
float y;
```

No entanto, dessa forma os dois valores ficam dissociados e, caso o programa manipule vários pontos, cabe ao programador evitar misturar a coordenada x de um ponto com a coordenada y de outro.

Para facilitar esse trabalho, a linguagem C oferece recursos para agrupar dados relacionados.

Uma estrutura em C permite agrupar diversas variáveis dentro de um único contexto.

No exemplo do ponto cartesiano, podemos definir uma estrutura chamada ponto que contém duas variáveis.

```
struct ponto {
   float x;
   float y;
};
```

Dessa forma, struct ponto passa a representar um novo tipo de dado e podemos declarar variáveis desse tipo.

```
struct ponto p;
```

Essa linha declara p como sendo uma variável do tipo struct ponto.

Os elementos de uma estrutura podem ser acessados por meio do operador ponto (.).

```
p.x = 10.0;
p.y = 5.0;
```

Manipulamos os elementos de uma estrutura da mesma forma que manipulamos variáveis simples. Podemos acessar seus valores, atribuir novos valores e também acessar seus endereços.

Exemplo completo:

```
#include <stdio.h>

struct ponto {
    float x;
    float y;
};

int main(void){

    struct ponto p;

    printf("Digite as coordenadas do ponto (x y): ");
    scanf("%f %f", &p.x, &p.y);

    printf("O ponto fornecido foi: (%.2f, %.2f)\n", p.x, p.y);

    return 0;
}
```

Definição de Novos Tipos com typedef

A palavra-chave typedef permite criar novos nomes para tipos de dados existentes.

Isso é útil para abreviar nomes de tipos ou tornar o código mais legível.

Exemplo:

```
typedef unsigned char UChar;
typedef float Vetor[4];
```

Agora podemos declarar variáveis usando esses novos nomes.

```
Vetor v;
```

Exemplo de uso:

```
v[0] = 3;
```

Neste caso:

UChar representa um tipo unsigned char
Vetor representa um vetor de quatro valores do tipo float

Uso de typedef com estruturas

Também podemos utilizar typedef para simplificar o uso de estruturas.

Exemplo:

```
struct ponto{
   float x;
   float y;
};

typedef struct ponto Ponto;
```

Neste caso:

struct ponto representa a estrutura com dois campos do tipo float
Ponto passa a ser um novo nome para esse tipo de estrutura

Também podemos fazer essa definição em um único comando.

```
typedef struct{
   float x;
   float y;
} Ponto;
```

Agora podemos declarar variáveis da seguinte forma:

```
Ponto p;
```

Uso de ponteiro em C ANSI

Um ponteiro é uma variável que armazena o endereço de memória de outra variável.

Uma analogia simples é pensar no endereço de uma casa. Para encontrar uma casa em uma cidade, precisamos conhecer o endereço completo. Da mesma forma, um ponteiro armazena o endereço onde um valor está localizado na memória.

Em C ANSI utilizamos o símbolo * para declarar um ponteiro.

Estrutura de declaração:

[tipo de variável] * [nome da variável];

Exemplo:

```
int *p;
```

Nesse caso, p é um ponteiro que pode armazenar o endereço de uma variável do tipo int.

Principais usos de ponteiros

Os ponteiros são utilizados para diversas atividades avançadas na programação, entre elas:

1. Permitir que funções modifiquem os argumentos recebidos por meio da passagem por referência
2. Passar vetores e strings de forma mais eficiente entre funções
3. Manipular elementos de vetores utilizando aritmética de ponteiros
4. Criar estruturas de dados complexas como listas encadeadas e árvores binárias
5. Alocar memória dinamicamente
6. Passar funções como parâmetros para outras funções

Exemplo de uso de ponteiros

```
#include <stdio.h>

int main(void){

    int a = 3;
    int *pa;
    printf("&a = %p \t a = %d\n", &a, a);
    pa = &a;
    printf("pa = %p \t *pa = %d\n", pa, *pa);
    *pa = 6;
    printf("a = %d\n", a);
    a = 5;
    printf("*pa = %d\n", *pa);
    return 0;
}
```

Nesse exemplo, o ponteiro pa armazena o endereço da variável a. Qualquer alteração feita através de *pa altera diretamente o valor da variável a.

Passando ponteiros para funções

Os ponteiros permitem alterar o valor de variáveis de forma indireta.

Em C, os parâmetros das funções são passados por valor, ou seja, a função recebe uma cópia da variável.
No entanto, se passarmos o endereço da variável, a função poderá modificar o valor original.

Exemplo de troca de valores entre duas variáveis.

```
#include <stdio.h>

void troca(int *px, int *py){

    int temp;

    temp = *px;
    *px = *py;
    *py = temp;
}

int main(void){

    int a = 5;
    int b = 7;

    troca(&a, &b);
    printf("a: %d  b: %d\n", a, b);

    return 0;
}
```

Nesse exemplo passamos os endereços das variáveis, permitindo que a função altere seus valores.

Passagem de vetores para funções

Quando passamos um vetor para uma função, na verdade estamos passando o endereço do primeiro elemento do vetor.
Isso significa que a função recebe um ponteiro para o vetor.
Exemplo de função que calcula a média de um vetor.

```
#include <stdio.h>

float media(int n, float *v){
```

```
    int i;
    float soma = 0.0;

    for(i = 0; i < n; i++){
        soma += v[i];
    }

    return soma / n;
}

int main(void){

    float v[10];
    float med;
    int i;

    for(i = 0; i < 10; i++){
        printf("Digite o valor %d: ", i + 1);
        scanf("%f", &v[i]);
    }
    med = media(10, v);
    printf("\nMedia = %.2f\n", med);

    return 0;
}
```

Nesse caso, a função media() recebe o endereço do vetor e pode acessar todos os seus elementos.

Exercícios:

1. Construa um programa em C ANSI que defina uma estrutura Pessoa (ex.: nome, idade, peso) e implemente duas funções: Pessoa lerPessoa(void) para preencher os campos via teclado e void imprimirPessoa(Pessoa p) para exibir os dados na tela.

2. Construa um programa em C ANSI que defina uma estrutura Animal com raça, cor, peso e sexo, armazene 10 animais em um vetor de estruturas, implemente uma função lerAnimal(Animal *a) e outra imprimirAnimal(const Animal *a), e ao final imprima todos os cadastros.

3. Construa um programa em C ANSI que defina uma estrutura Peca com código, nome, cor, peso e preço, implemente lerPeca(Peca *p) e imprimirPeca(Peca *p).

4. Construa um programa em C ANSI que leia a, b e c (com a != 0) e calcule as raízes da equação do 2º grau usando uma função bhaskara(float a, float b, float c, float *x1, float *x2) que grave os valores em *x1 e *x2, tratando delta < 0 com uma mensagem "sem raízes reais".

5. Construa um programa em C ANSI que leia o raio r e calcule a área do círculo em uma função void areaCirculo(float r, float *area) que grave o resultado em *area (usar pi = 3.141592), e no main imprima a área com duas casas decimais.

6. Construa um programa em C ANSI que leia dois inteiros a e b e implemente três funções void somar(int *a, int *b, int *res), void subtrair(int *a, int *b, int *res) e void multiplicar(int *a, int *b, int *res) que recebam entradas por ponteiro e gravem o resultado em *res, imprimindo os três resultados no main.

7. Construa um programa em C ANSI que leia um inteiro positivo n e calcule o fatorial usando uma função void fatorial(int n, double *fat) que grave o resultado em *fat, validando n >= 0 antes do cálculo.

8. Construa um programa em C ANSI que leia três inteiros a, b e c e ordene em ordem crescente usando uma função void ordenar3(int *a, int *b, int *c) que faça as trocas por referência e depois imprima os valores ordenados.

9. Implemente uma função em C ANSI chamada void imprimirNaPosicao(int linha, int coluna, char *msg) que utilize a biblioteca windows.h e a função SetConsoleCursorPosition() para posicionar o cursor na linha e coluna informadas e imprimir a mensagem, e no programa principal (main) solicite ao usuário os valores de linha, coluna e a mensagem para testar a função.

10. Implemente um programa em C ANSI que leia um nome em uma string e crie uma função void primeiraMaiuscula(char *nome) que altere o primeiro caractere para maiúsculo somente se for letra minúscula, recebendo a string por referência.

11. Implemente um programa em C ANSI que leia uma linha de texto e crie uma função void paraMaiusculas(char *texto) que converta todos os caracteres alfabéticos para maiúsculo, modificando o texto por referência.

12. Implemente uma função em C ANSI void media(const int *v, int n, float *m) que receba um vetor por ponteiro e seu tamanho n, calcule a média e grave em *m, validando n > 0, e no main leia n, preencha o vetor e imprima a média.